Cache locality is not enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan

Fabien André
Technicolor
fabien.andre@technicolor.com

Anne-Marie Kermarrec
Inria
anne-marie.kermarrec@inria.fr

Nicolas Le Scouarnec
Technicolor
nicolas.lescouarnec@technicolor.com

ABSTRACT

Nearest Neighbor (NN) search in high dimension is an important feature in many applications (e.g., image retrieval, multimedia databases). Product Quantization (PQ) is a widely used solution which offers high performance, i.e., low response time while preserving a high accuracy. PQ represents high-dimensional vectors (e.g., image descriptors) by compact codes. Hence, very large databases can be stored in memory, allowing NN queries without resorting to slow I/O operations. PQ computes distances to neighbors using cache-resident lookup tables, thus its performance remains limited by (i) the many cache accesses that the algorithm requires, and (ii) its inability to leverage SIMD instructions available on modern CPUs.

In this paper, we advocate that cache locality is not sufficient for efficiency. To address these limitations, we design a novel algorithm, PQ Fast Scan, that transforms the cache-resident lookup tables into small tables, sized to fit SIMD registers. This transformation allows (i) in-register lookups in place of cache accesses and (ii) an efficient SIMD implementation. PQ Fast Scan has the exact same accuracy as PQ, while having 4 to 6 times lower response time (e.g., for 25 million vectors, scan time is reduced from 74ms to 13ms).

1. INTRODUCTION

Nearest Neighbor (NN) search in high-dimensional spaces is an important feature in many applications including machine learning, multimedia databases and information retrieval. Multimedia data, such as audio, images or videos can be represented as feature vectors, characterizing their content. Finding a multimedia object similar to a given query object therefore involves representing the query object as a high-dimensional vector and finding its nearest neighbor in the feature vector space. While efficient methods exist to solve the NN search problem in low-dimensional spaces, the notorious *curse of dimensionality* challenges these solutions in high dimensionality. As the dimensionality increases, these methods are outperformed by bruteforce linear scans and large databases make this problem even more salient. To tackle this issue, the research community has focused on Approximate Nearest Neighbor (ANN) search, which aims at finding close enough vectors instead of the exact closest ones.

Locality Sensitive Hashing (LSH) [11, 8] has been proposed to solve the ANN problem. Yet, its significant storage overhead and important I/O cost limit its applicability to large databases. Recent advances in LSH-based approaches [19, 26] deal with these two issues but ANN search with these solutions still requires I/O operations. We focus on an alternative approach, named Product Quantization (PQ) [14, 27]. PQ is unique in that it stores database vectors as compact codes, allowing very large databases to be stored entirely in memory. In most cases, vectors can be represented by 8-byte codes, enabling a single commodity server equipped with 256 GB RAM to store 32 billion vectors in memory. Therefore, ANN search with PQ requires *no* I/O operations. The key idea behind PQ is to divide each vector into m distinct sub-vectors, and encode each sub-vector separately. To answer ANN queries, PQ computes the distance between the query vector and a large number of database vectors using cache-resident *lookup tables*. This process, known as *PQ Scan*, has a high CPU cost and is the focus of this paper.

As it relies on main memory, PQ Scan is able to scan hundreds of millions of database vectors per second. However, because it performs many table lookups, PQ Scan is not able to fully leverage the capabilities of modern CPUs. Its CPU cost therefore remains high, as reported in [19]. In particular, PQ Scan cannot leverage SIMD instructions, which are yet crucial for performance. In this paper, through the example of PQ Scan, we investigate why algorithms based on lookup tables cannot leverage SIMD instructions. We propose alternatives to cache-resident lookup tables, enabling better performance. Based on our findings, we design a novel algorithm, named PQ Fast Scan. PQ Fast Scan achieves 4-6× better performance than PQ Scan while returning the exact same results. More specifically, this paper makes the following contributions:

- We extensively analyze PQ Scan performance. Our study shows that sizing lookup tables to fit the L1 cache is not enough for performance. We also demonstrate that PQ Scan cannot be implemented efficiently with SIMD instructions, even using gather instructions available in the latest generations of Intel CPUs.
- We design PQ Fast Scan that addresses these issues. The key idea behind PQ Fast Scan is to build small

Proceedings of the VLDB Endowment, Vol. 9, No. 4

tables, sized to fit SIMD registers, that can be looked up using fast SIMD instructions. We use these small tables to compute lower bounds on distances and avoid unnecessary accesses to the cache-resident lookup tables. This technique allows pruning over 95% of L1 cache accesses and thus provides a significant performance boost. To build these small tables, we rely on: (i) grouping of similar vectors, (ii) computation of minimum tables and (iii) quantization of floating-point distances to 8-bit integers.

- We implement PQ Fast Scan on Intel CPUs and evaluate its performance on the public ANN_SIFT1B dataset of high-dimensional vectors. We analyze parameters that impact its performance, and experimentally show that it achieves a 4-6× speedup over PQ Scan.
- We discuss the application of the techniques used in PQ Fast Scan beyond the context of ANN search.

2. BACKGROUND

This section describes (i) how Product Quantization (PQ) is used to represent high-dimensional vectors by compact codes and (ii) the various steps involved in answering an ANN query with PQ.

2.1 Product Quantization

Product Quantization builds on vector quantizers to represent high-dimensional vectors by compact codes [14]. A vector quantizer is a function q that maps a d-dimensional vector x to a d-dimensional vector c_i belonging to a predefined set of vectors $\mathcal{C}$. Vectors c_i are called *centroids*, and the set of centroids $\mathcal{C}$, of size k, is the *codebook*.

$$q : \mathbb{R}^d \rightarrow \mathcal{C} = (c_0, \ldots, c_{k-1})$$
$$q(x) = \mathcal{C}[i] = c_i$$

We consider Lloyd-optimal quantizers which map vectors to their closest centroids and can be built using k-means [20].

$$\mathrm{q}(x) = \arg\min_{c_i \in \mathcal{C}} ||x - c_i||$$

Vector quantizers allow representing vectors by compact codes by using the index i of their closest centroid c_i as representative. This enables a 128-dimensional vector of floats (128×32 bits or 512 bytes of memory) to be represented by the index of its closest centroid (a single 64-bit integer or 8 bytes of memory).

$$\mathrm{code}(x) = i, \text{ such that } q(x) = \mathcal{C}[i]$$

To maintain the quantization error low, it is necessary to build a quantizer with a high number of centroids (e.g., $k = 2^{64}$ centroids). Yet, building such a quantizer is intractable both in terms of processing and memory requirements.

Product Quantization (PQ) addresses this issue by dividing an input vector x of dimensionality d into m distinct sub-vectors $u_j(x), 0 \leq j < m$, and quantizing each sub-vector separately using distinct lower complexity sub-quantizers. Each sub-vector $u_j(x)$ has a dimensionality $d^* = d/m$, where d is a multiple of m.

$$x = (\underbrace{x_0, \ldots, x_{d^*-1}}_{u_0(x)}, \ldots, \underbrace{x_{d-d^*}, \ldots, x_{d-1}}_{u_{m-1}(x)})$$

A product quantizer q_p uses m sub-quantizers to quantize the input vector x. Each sub-quantizer $q_j, 0 \leq j < m$ is a vector quantizer of dimensionality d^*, with a distinct codebook $\mathcal{C}_j$.

$$q_j : \mathbb{R}^{d^*} \rightarrow \mathcal{C}_j = (c_{j,0}, \ldots, c_{j,k^*-1})$$

A product quantizer q_p maps an input vector x of dimensionality d as follows:

$$\begin{aligned} q_p : \mathbb{R}^d &\rightarrow \mathcal{C}_0 \times \cdots \times \mathcal{C}_m \\ q_p(x) &= (q_0(u_0(x)), \ldots, q_{m-1}(u_{m-1}(x)) \\ &= (\mathcal{C}_0[i_0], \ldots, \mathcal{C}_m[i_m]) \end{aligned}$$

The product quantizer q_p can be used to represent an input vector x by a compact code, pqcode(x), by concatenating the indexes of the m centroids returned by the m sub-quantizers q_j:

$$\mathrm{pqcode}(x) = (i_0, \ldots, i_m),$$
$$\text{such that } q_p(x) = (\mathcal{C}_0[i_0], \ldots, \mathcal{C}_m[i_m])$$

We only consider product quantizers where sub-quantizers codebooks $\mathcal{C}_j$ have the same size k^*. The main advantage of product quantizers is that they are able to produce a large number of centroids k from sub-quantizers with a lower number of centroids k^*, such that $k = (k^*)^m$. For instance, a product quantizer with 8 sub-quantizers of 2^8 centroids has a total number of $k = (2^8)^8 = 2^{64}$ centroids. We focus on product quantizers with 2^{64} centroids as they offer a good tradeoff between complexity and quality for NN search [14].

We introduce the notation PQ $m\times \log_2(k^*)$ to designate a product quantizer with m sub-quantizers and k^* centroids per sub-quantizer. Thus, PQ 8×8 designates a product quantizer with 8 sub-quantizers and $2^8 = 256$ centroids per sub-quantizer. Any (m, k^*) configuration such that $m\times\log_2(k^*) = 64$ allows building a product quantizer with 2^{64} centroids. The m and k^* parameters impact (i) the complexity of learning the product quantizer, (ii) its accuracy and (iii) the memory representation of database vectors. Lower values of m, and thus higher values of k^* provide product quantizers with a lower quantization error at the price of an increased learning complexity. Database vectors are stored as pqcodes, which consist of m indexes of $\log_2(k^*)$ bits. Therefore, in the remainder of this paper, we refer to pqcodes stored in the database as database vectors, or simply vectors. Figure 1 shows the memory representation of 6 database vectors, $p, \cdots, u$ encoded with a PQ 8×8 quantizer. Each vector is composed of 8 indexes of 8 bits.

	0							7
p	01	03	02	05	06	09	04	08
q	3f	11	21	00	01	f2	12	11
r	04	0c	0e	1a	f1	0f	a9	17
s	f6	ff	f6	f0	23	0b	b6	2f
t	37	1a	21	00	32	8b	e9	03
u	f5	fc	ff	f1	46	33	cf	2c
	⋮							

Figure 1: Database vectors in memory

2.2 ANN Search with Product Quantization

Nearest Neighbor Search requires computing distances between vectors stored in the database and a query vector y. We consider squared distances as they avoid a square root

Algorithm 1 Nearest Neighbor Search with PQ

```
1: function NNS(y, database)
2:     part ← INDEX_GET_PARTITION(y, database)   ▷ Step 1
3:     D ← COMPUTE_DISTANCE_TABLES(y)            ▷ Step 2
4:     PQSCAN(D, part)                           ▷ Step 3
5: end function

6: function PQSCAN(D, part)
7:     min ← ∞
8:     pos ← 0
9:     for i ← 0 to |part| − 1 do
10:        p ← part_i                  ▷ i^th database vector
11:        d ← PQDISTANCE(p, D)
12:        if d < min then
13:            min ← d
14:            pos ← i
15:        end if
16:    end for
17:    return min, pos
18: end function

19: function PQDISTANCE(p, D)
20:    d ← 0
21:    for j ← 0 to m − 1 do
22:        index ← p[j]            ▷ index of the j^th centroid
23:        d ← d+D_j[index]
24:    end for
25:    return d
26: end function
```

computation while preserving the order. The Asymmetric Distance Computation (ADC) method allows computing the distance between a query vector y and a database vector p, without quantizing the query vector [14]. ADC approximates the distance between the query vector y and a database vector p by:

$$\tilde{\mathrm{d}}(\mathrm{p},\mathrm{y}) = \sum_{j=0}^{m-1} \mathrm{d}(u_j(y), \mathcal{C}_j[p[j]]) \quad (1)$$

where $\mathrm{d}(u_j(y), \mathcal{C}_j[p[j]])$ is the squared distance between the j^{th} sub-vector of the query vector y and $\mathcal{C}_j[p[j]]$, the j^{th} centroid associated with the database vector p.

To allow fast ANN queries, the IVFADC [14] system has been proposed. This system adds an inverted index (IVF) on top of the product quantizer and ADC. The index is built from a basic vector quantizer, named *coarse quantizer*. Each Voronoi cell of the coarse quantizer forms a *partition* of the database. Answering an ANN query with IVFADC involves three steps (Algorithm 1):

1. Selecting a partition of the database using the index. The selected partition corresponds to the Voronoi cell of the coarse quantizer where the query vector lies.
2. Computing distance tables, which are used to speed up ADC computations.
3. Scanning the partition, i.e., computing the ADC between the query vector and all vectors of the partition. We name this step PQ Scan.

PQ achieves high recall by scanning a large number of vectors. The selected partition typically comprises thousands

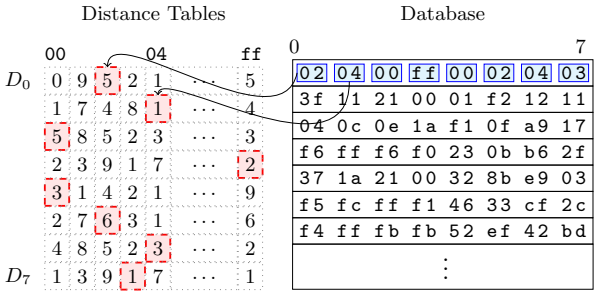

Figure 2: PQ distance computation

to millions of vectors, depending on the database size and the index parameters. We focus on very large databases and partitions exceeding 3 million vectors. In this case, Step 1 and 2 account for less than 1% of the CPU time while Step 3 uses more than 99% of the CPU time.

Once a partition has been selected, m distance tables, D_j, $0 \leq j < m$, are computed (Step 2). These distance tables are specific to a given query vector. Each D_j distance table is composed of the distance between the j^{th} sub-vector of the query vector y and every centroid of the j^{th} sub-quantizer:

$$D_j = \left(d\left(u_j(y), \mathcal{C}_j[0]\right), \cdots, d\left(u_j(y), \mathcal{C}_j[k^*-1]\right)\right) \quad (2)$$

We omitted the definition of COMPUTE_DISTANCE_TABLES in Algorithm 1 for the sake of clarity but it would correspond to an implementation of Equation (2). Using these distance tables, the ADC equation (1) can be rewritten as:

$$\tilde{\mathrm{d}}(\mathrm{p},\mathrm{y}) = \sum_{j=0}^{m-1} D_j[p[j]] \quad (3)$$

PQ Scan (Step 3) iterates over all vectors (Algorithm 1, line 9) of the selected partition, and computes the ADC between the query vector and every vector of the partition using the PQDISTANCE function (Algorithm 1, line 11). The PQDISTANCE function is an implementation of Equation (3). Figure 2 shows the pqdistance computation between the query vector and the first database vector. The first centroid index ($p[0] = 02$) is used to look up a value in the first distance table (D_0), the second centroid index ($p[1] = 04$) is used to lookup a value in the second distance table (D_1) etc. All looked up values are then added to compute the final distance.

3. PQ SCAN LIMITATIONS

In this section, we show that despite its low apparent complexity, PQ Scan cannot be implemented efficiently on CPUs. We identify two fundamental bottlenecks that limit its performance: (i) the many cache accesses it performs and (ii) the impossibility to implement it efficiently using SIMD. Identification of these bottlenecks is key to designing our novel algorithm that overcomes PQ Scan limitations.

3.1 Memory Accesses

As PQ Scan parallelizes naturally over multiple queries by running each query on a different core, we focus on single-core performance. PQ Scan computes the pqdistance between the query vector and each database vector, which accounts for almost all CPU cycles consumed. The number

of operations required for each pqdistance computation depends on the m parameter of the product quantizer. Each pqdistance computation involves:

- m memory accesses to load centroid indexes $p[j]$ (Algorithm 1, line 22) [*mem1*]
- m memory accesses to load D_j[index] values from distance tables (Algorithm 1, line 23) [*mem2*]
- m additions (Algorithm 1, line 23)

We distinguish between two types of memory accesses: *mem1*, accesses to centroid indexes and *mem2*, accesses to distance tables as they may hit different cache levels. We analyze the locality of these two types of memory accesses. *Mem1* accesses always hits the L1 cache thanks to *hardware prefetchers* included in modern CPUs. Indeed, hardware prefetchers are able to detect sequential memory accesses and prefetch data to the L1 cache. We access p[j] values sequentially, i.e., we first access p[0] where p is the first database vector, then p[1] until p[m-1]. Next, we perform the same accesses on the second database vector, until we have iterated on all database vectors.

The cache level accessed by the *mem2* accesses depends on the m and k^* parameters of the product quantizer. Indeed, the size of distance tables, which impacts the cache level they can be stored in, is given by $m \times k^* \times$ sizeof(`float`). To obtain a product quantizer with 2^{64} centroids, which is important for accuracy, $m \times \log_2(k^*) = 64$. Therefore, smaller m values lead to less memory accesses and additions but imply higher k^* values and thus larger distance tables, which are stored in higher cache levels. Table 1 summarizes the properties of the different cache levels as well as the product quantizer configurations of which they can store the distance tables.

Table 1: Cache level properties (Nehalem-Haswell)

	L1	L2	L3
Latency (cycles)	4-5	11-13	25-40
Size	32KiB	256KiB	2-3MiB × nb cores
PQ Configurations	PQ 16×4 PQ 8×8		PQ 4×16

PQ 16×4 is not interesting because it requires more memory accesses than PQ 8×8 and distance tables can be stored in the L1 cache for both configurations. PQ 4×16 requires two times less memory accesses than PQ 8×8, but PQ 4×16 distance tables are stored in the L3 cache which has a 5 times higher latency than the L1 cache. Overall, PQ 8×8 provides the best performance tradeoff, and is the most commonly used configuration in the literature [14, 27, 4, 10, 21]. Thus, from now on, we focus exclusively on PQ 8×8.

We use performance counters to analyze the performance of different PQ Scan implementations experimentally (Figure 3). For all implementations, the number of cycles with pending load operations (cycles w/ load) is almost equal to the number of cycles, which confirms that PQ Scan is a memory-intensive algorithm. We also measured the number of L1 cache misses (not shown on Figure 3). L1 cache misses represent less than 1% of memory accesses for all implementations, which confirms that both *mem1* and *mem2* accesses hit the L1 cache. The naive implementation of PQ Scan (Algorithm 1) performs 16 L1 loads per scanned vector: 8 *mem1* accesses and 8 *mem2* accesses. The authors of [14] distribute the libpq library[1], which includes an optimized implementation of PQ Scan. We obtained a copy of libpq under a commercial licence. Rather than loading 8 centroid indexes of 8 bits each (*mem1* accesses), the libpq implementation of PQ Scan loads a 64-bit word into a register, and performs 8-bit shifts to access individual centroid indexes. This allows reducing the number of *mem1* accesses from 8 to 1. Therefore, the libpq implementation of PQ Scan performs 9 L1 loads per scanned vector: 1 *mem1* access and 8 *mem2* accesses. However, overall, the libpq implementation is slightly slower than the naive implementation on our Haswell processor. Indeed, the increase in the number of instructions offsets the increase in IPC (Instructions Per Cycle) and the decrease in L1 loads.

3.2 Inability to Leverage SIMD Instructions

In addition to cache accesses, PQ Scan requires m additions per pqdistance computation. We evaluate the applicability of SIMD instructions to reduce the number of instructions and CPU cycles devoted to additions. SIMD instructions perform the same operation, e.g., additions, on multiple data elements in one instruction. To do so, SIMD instructions operate on wide registers. SSE SIMD instructions operate on 128-bit registers, while more recently introduced AVX SIMD instructions operate on 256-bit registers [2]. SSE instructions can operate on 4 floating-point *ways* (4×32 bits, 128 bits) while AVX instructions can operate on 8 floating-point *ways* (8×32 bits, 256 bits). In this subsection, we consider AVX instructions as they provided the best results in our experiments. We show that PQ Scan structure prevents an efficient use of SIMD instructions.

To enable the use of fast *vertical* SIMD additions, we compute the pqdistance between the query vector and 8 database vectors at a time, designated by the letters a to h. We still issue 8 addition instructions, but each instruction involves 8 different vectors, as shown on Figure 4. Overall, the number of instructions devoted to additions is divided by 8. However, the gain in cycles brought by the use of SIMD additions is offset by the need to set the ways of SIMD registers one by one. SIMD processing works best when all values in all ways are *contiguous* in memory and can be loaded in one instruction. Because they were looked up in a table, $D_0[a[0]], D_0[b[0]], \cdots, D_0[h[0]]$ values are not contiguous in memory. We therefore need to insert $D_0[a[0]]$ in the first way of the SIMD register, then $D_0[b[0]]$ in the second way etc. In addition to memory accesses, doing so requires many SIMD instructions, some of which have high latencies. Overall, this offsets the benefit provided by SIMD additions. This explains why algorithms relying on lookup tables, such as PQ Scan, hardly benefit from SIMD processing. Figure 3 shows that the AVX implementation of PQ Scan requires slightly less instructions than the naive implementation, and is only marginally faster.

To tackle this issue, Intel introduced a `gather` SIMD instruction in its latest architecture, Haswell [2]. Given an SIMD register containing 8 indexes and a table stored in memory, `gather` looks up the 8 corresponding elements from the table and stores them in a register, in just one instruction. This avoids having to use many SIMD instructions to set the 8 ways of SIMD registers. Figure 5 shows how `gather`

[1] `http://people.rennes.inria.fr/Herve.Jegou/projects/ann.html`

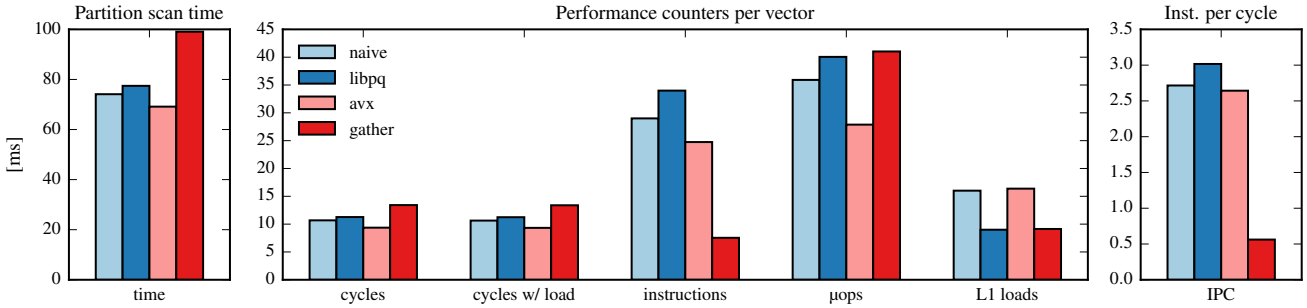

Figure 3: Scan times and performance counters for 4 implementations of PQ Scan (25M vectors)

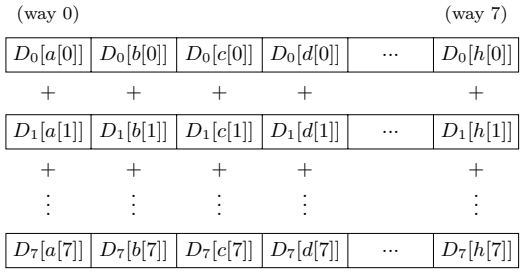

Figure 4: PQ Scan with SIMD vertical adds

can be used to look up 8 values in the first distance table (D_0). To efficiently use `gather`, we need $a[0], \cdots, h[0]$ to be stored contiguously in memory, so that they can be loaded in one instruction. To do so, we transpose the memory layout of the database presented in Figure 1. We store the first components of 8 vectors contiguously ($a[0], \cdots, h[0]$), followed by the second components of the same 8 vectors ($a[1], \cdots, h[1]$) etc., instead of storing all components of the first vector ($a[0], \cdots, a[7]$), followed by the components of the second vector ($b[0], \cdots, b[7]$). This also allows to reduce the number of *mem1* accesses from 8 to 1, similarly to the libpq implementation.

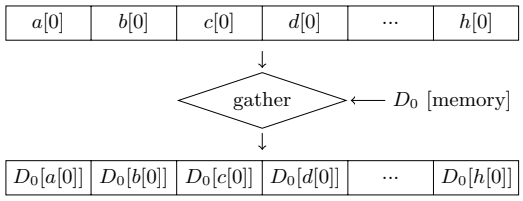

Figure 5: SIMD gather operation

However, the gather implementation of PQ Scan is slower than the naive version (Figure 3), which can be explained by several factors. First, even if it consists of only one instruction, `gather` performs 1 memory access for each element it loads, which implies suffering memory latencies. Second, at the hardware level, `gather` executes 34 µops[2] (Table 2) where most instructions execute only 1 µop. Figure 3 shows that the gather implementation has a low instructions count

[2]Micro-operations (µops) are the basic operations executed by the processor. Instructions are sequences of µops.

Table 2: Instruction properties (Haswell)

Inst.	Lat.	Through.	µops	# elem	elem size
`gather`	18	10	34	no limit	32 bits
`pshufb`	1	0.5	1	16	8 bits

but a high µops count. For other implementations, the number of µops is only slightly higher than the number of instructions. It also has a high latency of 18 cycles and a throughput of 10 cycles (Table 2), which means it is necessary to wait 10 cycles to pipeline a new `gather` instruction after one has been issued. This translates into poor pipeline utilization, as shown by the very low IPC of the gather implementation (Figure 3). In its documentation, Intel acknowledges that `gather` instructions may only bring performance benefits in specific cases [1] and other authors reported similar results [13].

4. PQ FAST SCAN

In this section, we present PQ Fast Scan, a novel algorithm we design that overcomes the limitations of PQ Scan. PQ Fast Scan performs less than 2 L1 cache accesses per scanned vector and allows additions to be implemented efficiently using SIMD instructions. By design, PQ Fast Scan returns exactly the same results as PQ Scan with a PQ 8×8 quantizer while performing 4-6 times faster.

4.1 Description

The key idea behind PQ Fast Scan is to use *small tables*, sized to fit SIMD registers instead of the cache-resident distance tables. These small tables are used to compute *lower bounds* on distances, without accessing the L1 cache. Therefore, lower bounds computations are fast. In addition, they are implemented using SIMD additions, further improving performance. We use lower-bound computations to prune slow pqdistance computations, which access the L1 cache and cannot benefit from SIMD additions. Figure 6 shows the processing steps applied to every database vector p. The $\otimes$ symbol means we discard the vector p and move to the next database vector. The *min* value is the distance of the query vector to the current nearest neighbor. Our experimental results on SIFT data show that PQ Fast Scan is able to prune 95% of pqdistance computations.

To compute lower bounds, we need to look up values in small tables stored in SIMD registers. To do so, we use the `pshufb` instruction, which is key to PQ Fast Scan performance. Similarly to `gather`, `pshufb` looks up values in a